

PassengersStart

January 13, 2026

0.1 ## 1) Import & Display Options

```
[1]: import numpy as np
import pandas as pd

# Viz
from matplotlib import pyplot as plt
import seaborn as sns

# Pretty plots
plt.style.use('seaborn-v0_8')
sns.set_context('notebook')

# Pandas display
pd.set_option('display.max_columns', 50)
pd.set_option('display.width', 120)
```

0.2 ## 2) Load Titanic Passengers Data

```
[2]: df = pd.read_csv('./data/Titanic2026.csv')
print('Rows, Columns:', df.shape)
df.head()
```

Rows, Columns: (1317, 9)

```
[2]:
```

	ID	LastName	FirstName	Title	Gender	Boarded	Class	Age
								Survived
0	1	Ab i-Al-Munà	Nasif Qasim	Mr	M	Cherbourg	3rd	26.0
								Yes
1	2	Abbing	Anthony	Mr	M	Southampton	3rd	42.0
								No
2	3	Abbott Rhoda Mary	"Rosa"	Mrs	F	Southampton	3rd	39.0
								Yes
3	4	Abbott Rossmore	Edward	Mr	M	Southampton	3rd	16.0
								No
4	5	Abbott Eugene Joseph		Mr	M	Southampton	3rd	14.0
								No

0.3 ## 3) Data and Type Check

```
[3]: # print the types of each feature
```

```
ID                int64
LastName          object
FirstName         object
Title            object
Gender           object
Boarded          object
Class            object
Age              float64
Survived         object
dtype: object
```

```
[4]: # Print the df columns
```

```
Index(['ID', 'LastName', 'FirstName', 'Title', 'Gender', 'Boarded', 'Class',
      'Age', 'Survived'], dtype='object')
```

```
[5]: # Unique values for key features
```

```
### NOTE ### You don't have to do it in this exact output - can you get the
↳ answers?
```

```
Gender → ['M' 'F']
```

```
Boarded → ['Cherbourg' 'Southampton' 'Belfast' 'Queenstown']
```

```
Class → ['3rd' '2nd' '1st']
```

```
Survived → ['Yes' 'No']
```

```
Title → ['Mr' 'Mrs' 'Miss' 'Master' 'Colonel' 'Mme.' 'Reverend' 'Dr' 'Major'
        'Father' 'Captain' 'Sir' 'Lady' 'Señora' 'Mlle' 'Ms' 'Doña' 'Señor'
        'Countess' 'Don.']
```

```
[6]: # Extended to include counts:
```

```
### NOTE ### You don't have to do it in this exact output - can you get the
↳ answers?
```

```
=== Gender ===
```

	Gender_count	Gender_percent
Gender		
M	851	64.6
F	466	35.4

=== Boarded ===

	Boarded_count	Boarded_percent
Boarded		
Southampton	912	69.2
Cherbourg	272	20.7
Queenstown	123	9.3
Belfast	10	0.8

=== Class ===

	Class_count	Class_percent
Class		
3rd	709	53.8
1st	324	24.6
2nd	284	21.6

=== Survived ===

	Survived_count	Survived_percent
Survived		
No	833	63.2
Yes	484	36.8

=== Title ===

	Title_count	Title_percent
Title		
Mr	765	58.1
Miss	257	19.5
Mrs	198	15.0
Master	61	4.6
Dr	8	0.6
Father	6	0.5
Colonel	4	0.3
Señora	3	0.2
Major	2	0.2
Reverend	2	0.2
Mlle	2	0.2
Mme.	1	0.1
Sir	1	0.1
Captain	1	0.1
Lady	1	0.1
Ms	1	0.1
Doña	1	0.1
Señor	1	0.1
Countess	1	0.1

Don. 1 0.1

0.4 ## 4) Check for and Dealing with NaNs

- Delete the rows?
- Impute values?

```
[7]: # 1) Count of nulls per column

# 2) Percent of nulls per column (only columns that have any nulls)

# 3) Preview of rows that contain at least one null
```

Nulls per column:

	null_count
Age	2
ID	0
LastName	0
Title	0
FirstName	0
Gender	0
Boarded	0
Class	0
Survived	0

Columns with any nulls (percent):

	null_percent
Age	0.2

Rows with at least one null: 2 (showing up to 20)

	ID	LastName	FirstName	Title	Gender	Boarded	Class	Age	Survived
439	440	Gheorgheff	Stanio	Mr	M	Cherbourg	3rd	NaN	No
675	676	Kraeff	Theodor	Mr	M	Cherbourg	3rd	NaN	No

```
[8]: # Delete rows OR define the subgroup to determine a realistic median or mean to
    ↪impute:

# Define subgroup: Male, 3rd class, Cherbourg, Survived = No
```

Subgroup Count: 54

```
[24]: # Show the stats for each defined subgroup
```

```
Subgroup size: 54
Mean Age: 25.09
Median Age: 25.00
```

```
[10]: # Impute missing Age with median (common practice)
```

0.5 ## 5) Basic Cleaning:

- trim spaces
- standardize cases
- fix numerics AS *numeric*

```
[11]: # Strip leading/trailing spaces in all object columns
obj_cols = df.select_dtypes(include='object').columns
df[obj_cols] = df[obj_cols].apply(lambda s: s.str.strip())
```

```
[12]: # Standardize categorical case (title case for names, upper/lower for codes)
for c in ['Gender', 'Boarded', 'Class', 'Survived', 'Title']:
    if c in df.columns:
        df[c] = df[c].str.strip()
```

```
[14]: # Age to numeric (coerce non-numeric to NaN)
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')
```

```
[15]: # Quick check of missing values (post-clean) (i.e. if any Ages were changed to
      ↪ NaN with coerce they would show here)
```

```
[15]: ID          0
      LastName     0
      FirstName    0
      Title        0
      Gender       0
      Boarded      0
      Class        0
      Age          0
      Survived     0
      dtype: int64
```

0.6 ## 6) Create Helper Features (for Analysis)

- Child under 12
- Passenger groupings
- Encode to binary
 - Survival (Yes -> 1, No -> 0)
- Encode to ordinal
 - Class (1st -> 1, 2nd -> 2, etc)
- Encode to short codes
 - Boarded (Cherbourg -> C, Southampton -> S, etc)

```
[16]: # Is child (under 12)? (Common Titanic framing)
# See output from code below df.head() below for helper feature names
```

```
[17]: df.head()
```

```
[17]:   ID      LastName      FirstName Title Gender      Boarded Class  Age
Survived IsChild \
0    1  Ab i-Al-Munà      Nasif Qasim   Mr      M      Cherbourg   3rd  26.0
Yes      False
1    2      Abbing      Anthony      Mr      M      Southampton   3rd  42.0
No      False
2    3      Abbott  Rhoda Mary "Rosa"  Mrs      F      Southampton   3rd  39.0
Yes      False
3    4      Abbott  Rossmore Edward   Mr      M      Southampton   3rd  16.0
No      False
4    5      Abbott      Eugene Joseph  Mr      M      Southampton   3rd  14.0
No      False
```

	NameFull	Survived_bin	Class_num	Embark
0	Mr Nasif Qasim Ab i-Al-Munà	1	3	C
1	Mr Anthony Abbing	0	3	S
2	Mrs Rhoda Mary "Rosa" Abbott	1	3	S
3	Mr Rossmore Edward Abbott	0	3	S
4	Mr Eugene Joseph Abbott	0	3	S

0.7 ## 7) Basic Survival Stats

```
[18]: ### NOTE ### You don't have to do it in this exact output - can you get the
↳ answers?
```

Overall survival rate: 36.75%

By Gender:

Gender	
F	69.3%
M	18.9%

Name: Survived_bin, dtype: object

By Class:

Class	
1st	62.0%
2nd	39.1%
3rd	24.3%

Name: Survived_bin, dtype: object

By Boarding Port:

Boarded

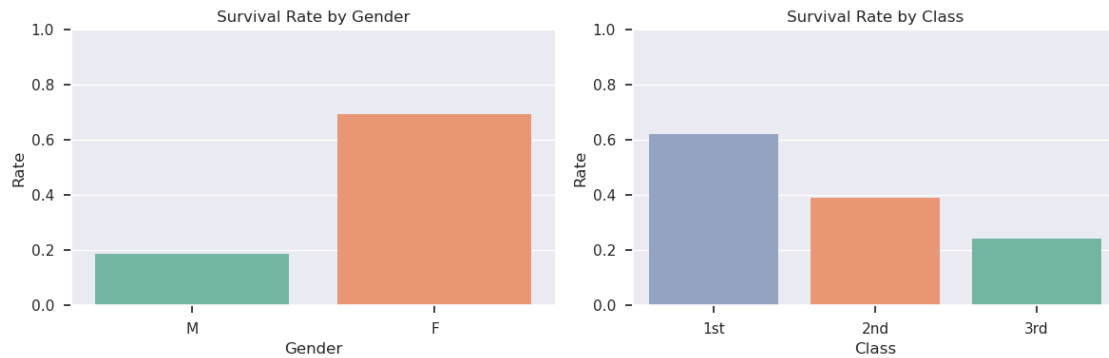
```
Cherbourg      55.9%
Southampton    32.1%
Queenstown     31.7%
Belfast        0.0%
Name: Survived_bin, dtype: object
```

```
Children vs Adults:
  IsChild
False    35.3%
True     52.8%
Name: Survived_bin, dtype: object
```

0.8 ## 8) Visualize Survival Rate

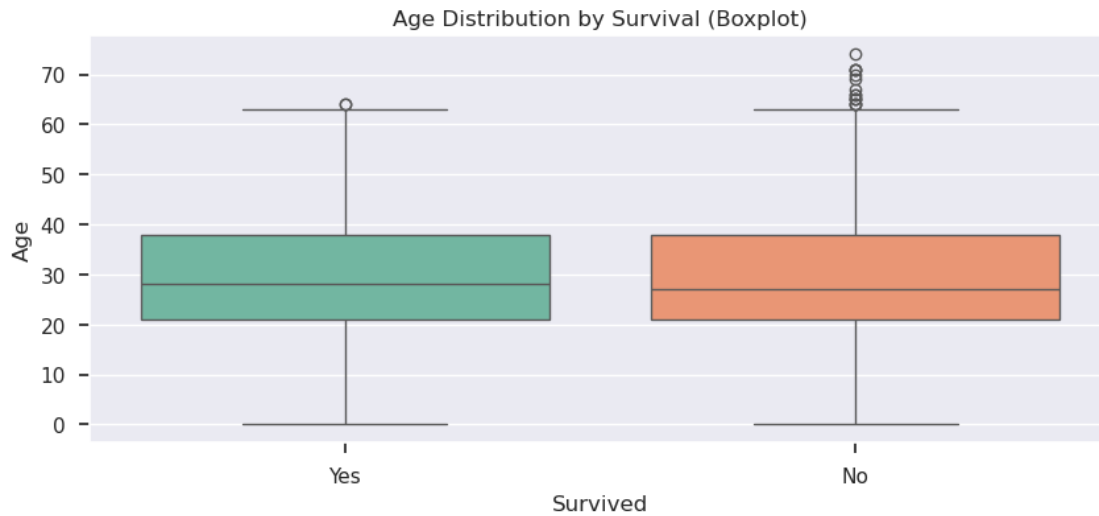
by Gender and Class

[19]:

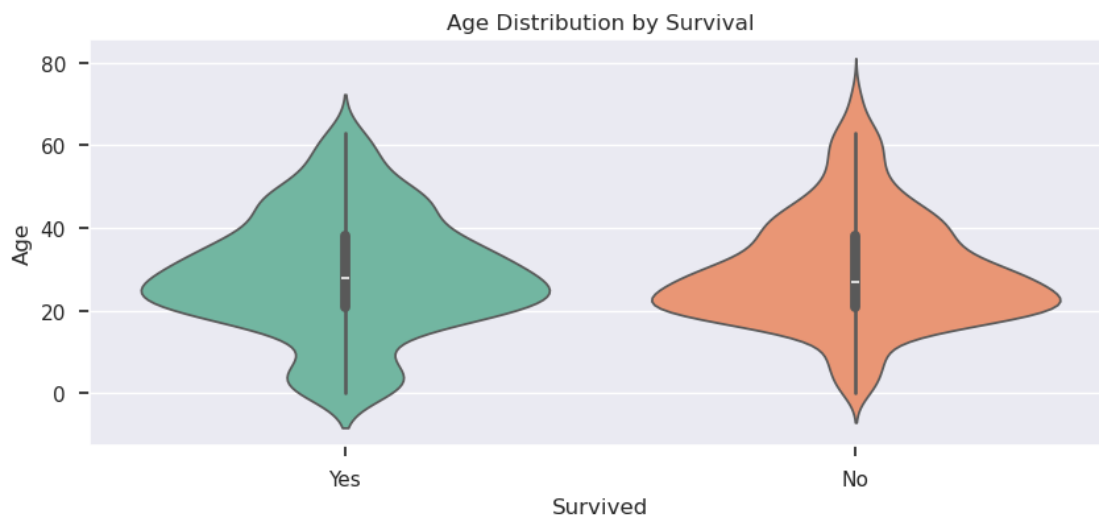


0.9 ## 9) Age Distributions

[20]:



[21]: `# Violin plot alternative`



0.10 ## 10) Survival by Key Features

Utilitizing Crosstabs

[22]: `### NOTE ### You don't have to do it in this exact output - can you get the answers?`

Survival rate (mean) by Gender × Class:

Class	1st	2nd	3rd
Gender			

F	0.965	0.821	0.449
M	0.344	0.135	0.152

Survival rate (mean) by Boarded × Class:

Class	1st	2nd	3rd
Boarded			
Belfast	0.000	0.000	NaN
Cherbourg	0.715	0.538	0.343
Queenstown	0.667	0.286	0.310
Southampton	0.555	0.388	0.206

Survival rate (mean) for Children × Class:

Class	1st	2nd	3rd
IsChild			
False	0.618	0.34	0.225
True	0.800	1.00	0.380